BITS PILANI, DUBAI CAMPUS
DUBAI INTERNATIONAL ACADEMIC CITY, DUBAI
SECOND SEMESTER 2025 - 2026

Data Structures & Algorithms (CS F211)

```c
/*
Name: Kirtika
Roll No: 2023A7PS0219U
Course: CS F211 - Data Structures & Algorithms
Lab Exercise 5: Linear Search and Binary Search
*/

#include <stdio.h>
#include <sys/time.h>

#define ARRAY_SIZE 10000

typedef struct {
    int index;
    long long iterations;
    long long time_us;
} SearchResult;

static long long current_time_us(void) {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return (long long)tv.tv_sec * 1000000LL + (long long)tv.tv_usec;
}

static SearchResult linear_search(const float arr[], int size, float key, int
print_trace) {
    SearchResult result = {-1, 0, 0};
    long long start = current_time_us();

    for (int i = 0; i < size; i++) {
        result.iterations++;
        if (print_trace) {
            printf("LS checks index %d (value = %.0f)\n", i, arr[i]);
        }

        if (arr[i] == key) {
            result.index = i;
            break;
        }
    }

    result.time_us = current_time_us() - start;
    return result;
```

```c
}

static SearchResult binary_search(const float arr[], int size, float key, int
print_trace) {
    SearchResult result = {-1, 0, 0};
    int left = 0;
    int right = size - 1;
    long long start = current_time_us();

    while (left <= right) {
        int mid = left + (right - left) / 2;
        result.iterations++;

        if (print_trace) {
            printf("BS checks index %d (value = %.0f)\n", mid, arr[mid]);
        }

        if (arr[mid] == key) {
            result.index = mid;
            break;
        }

        if (arr[mid] < key) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    result.time_us = current_time_us() - start;
    return result;
}

static void print_result(const char *algorithm, float key, SearchResult result) {
    printf("\n%s Result for key %.0f\n", algorithm, key);
    printf("Position (index): %d\n", result.index);
    printf("Iterations      : %lld\n", result.iterations);
    printf("Search Time     : %lld micro-secs\n", result.time_us);
}

static int log2_upper_bound(int n) {
    int steps = 0;
    while (n > 0) {
        steps++;
        n /= 2;
    }
    return steps;
}

int main(void) {
```

```c
    float arr[ARRAY_SIZE];

    for (int i = 0; i < ARRAY_SIZE; i++) {
        arr[i] = (float)i;
    }

    float user_key;
    printf("Enter search key value: ");
    if (scanf("%f", &user_key) != 1) {
        printf("Invalid input.\n");
        return 1;
    }

    SearchResult ls_user = linear_search(arr, ARRAY_SIZE, user_key, 0);
    SearchResult bs_user = binary_search(arr, ARRAY_SIZE, user_key, 0);

    print_result("Linear Search", user_key, ls_user);
    print_result("Binary Search", user_key, bs_user);

    printf("\nWorst-case style checks (as requested):\n");

    float test_keys[] = {9999.0f, 50000.0f};
    int test_count = (int)(sizeof(test_keys) / sizeof(test_keys[0]));

    for (int i = 0; i < test_count; i++) {
        SearchResult ls_test = linear_search(arr, ARRAY_SIZE, test_keys[i], 0);
        SearchResult bs_test = binary_search(arr, ARRAY_SIZE, test_keys[i], 0);

        print_result("Linear Search", test_keys[i], ls_test);
        print_result("Binary Search", test_keys[i], bs_test);
    }

    int bs_bound = log2_upper_bound(ARRAY_SIZE);
    printf("\nComplexity verification:\n");
    printf("Linear Search worst-case iterations should be close to n = %d.\n",
ARRAY_SIZE);
    printf("Binary Search worst-case iterations should be close to log2(n), about
<= %d iterations.\n", bs_bound);

    return 0;
}
```

OUTPUT

```
manavaryasingh@MacBook-Pro-5 kritika-dsa %  cd "/Users/manavaryasingh/kritika-dsa" && gcc "week 5/ex5_search_linear_binary.c" -o "week
week 5/ex5" << 'EOF'
9999
EOF
Enter search key value:
Linear Search Result for key 9999
Position (index): 9999
Iterations      : 10000
Search Time     : 20 micro-secs

Binary Search Result for key 9999
Position (index): 9999
Iterations      : 14
Search Time     : 0 micro-secs

Worst-case style checks (as requested):

Linear Search Result for key 9999
Position (index): 9999
Iterations      : 10000
Search Time     : 20 micro-secs

Binary Search Result for key 9999
Position (index): 9999
Iterations      : 14
Search Time     : 0 micro-secs

Linear Search Result for key 50000
Position (index): -1
Iterations      : 10000
Search Time     : 20 micro-secs

Binary Search Result for key 50000
Position (index): -1
Iterations      : 14
Search Time     : 0 micro-secs

Complexity verification:
Linear Search worst-case iterations should be close to n = 10000.
Binary Search worst-case iterations should be close to log2(n), about <= 14 iterations.
manavaryasingh@MacBook-Pro-5 kritika-dsa %
```